

DAA Lab Codes

1 Fractional Knapsack in C

```
#include <stdio.h>

void swap(float *a, float *b){
    float temp;
    temp=*a;
    *a=*b;
    *b=temp;
}

void bubbleSort(int n, float p[], float w[], float ratio[]){
    int i,j;

    for(i=0;i<n-1;i++){
        for(j=0;j<n-i-1;j++){
            if(ratio[j]<ratio[j+1]){
                swap(&ratio[j],&ratio[j+1]);
                swap(&p[j],&p[j+1]);
                swap(&w[j],&w[j+1]);
            }
        }
    }
}

int main(){
    int n,i;

    float p[20],w[20],ratio[20];
    float x[20], capacity, maxProfit=0;

    printf("Enter the no. of items: ");
    scanf("%d",&n);

    printf("Enter knapsack capacity: ");
```

```

scanf("%f",&capacity);

for(i=0;i<n;i++){

    printf("\nEnter profit and weight of item %d: ",i+1)
        ;

    scanf("%f %f",&p[i],&w[i]);

    ratio[i]=p[i]/w[i];
}

printf("\nOriginal Table\n");

printf("-----\n");
printf("Item\tProfit\tWeight\tRatio\n");
printf("-----\n");

for(i=0;i<n;i++){

    printf("%d\t%.0f\t%.0f\t%.2f\n",
        i+1,p[i],w[i],ratio[i]);
}

printf("-----\n");

bubbleSort(n,p,w,ratio);

printf("\nSorted Table\n");

printf("-----\n");
printf("Item\tProfit\tWeight\tRatio\n");
printf("-----\n");

for(i=0;i<n;i++){

    printf("%d\t%.0f\t%.0f\t%.2f\n",
        i+1,p[i],w[i],ratio[i]);
}

printf("-----\n");

for(i=0;i<n;i++){
    x[i]=0.0;
}

for(i=0;i<n;i++){

    if(w[i]<=capacity){

```

```

        x[i]=1.0;
        maxProfit+=p[i];
        capacity-=w[i];
    }

    else{

        x[i]=capacity/w[i];
        maxProfit+=p[i]*x[i];
        break;
    }
}

printf("\nSolution Vector:\n");

for(i=0;i<n;i++){

    if(i<(n-1))
        printf("%.1f, ",x[i]);
    else
        printf("%.1f",x[i]);
}

printf("\n\nMaximum Profit: %.2f",maxProfit);

return 0;
}

```

2 BFS in Python

```

import networkx as nx
import matplotlib.pyplot as plt
from collections import deque

g = nx.Graph()

e = int(input("Enter number of edges: "))

for _ in range(e):

    u, v = map(int, input().split())

    g.add_edge(u, v)

start = int(input("Enter starting vertex: "))

visited = set()
q = deque([start])

```

```

print("BFS Traversal:")

while q:
    node = q.popleft()

    if node not in visited:
        print(node, end=" ")
        visited.add(node)

        for neighbor in g[node]:
            if neighbor not in visited:
                q.append(neighbor)

pos = nx.spring_layout(g)

nx.draw(g, pos, with_labels=True)

plt.show()

```

3 Kruskal's Algorithm in Python

```

import networkx as nx
import matplotlib.pyplot as plt

g = nx.Graph()

e = int(input("Enter number of edges: "))

print("Enter u v w:")

for _ in range(e):
    u, v, w = map(int, input().split())
    g.add_edge(u, v, weight=w)

mst = nx.minimum_spanning_tree(g)

cost = 0

print("\nEdges in MST:")

for u, v, w in mst.edges(data=True):

```

```

        print(u, "--", v, "=", w['weight'])

        cost += w['weight']

print("\nMinimum Cost =", cost)

pos = nx.spring_layout(mst)

labels = nx.get_edge_attributes(mst, 'weight')

nx.draw(mst, pos, with_labels=True)

nx.draw_networkx_edge_labels(mst,
                             pos,
                             edge_labels=labels)

plt.show()

```

4 Graph Perimeter Method 1 (Using i;j)

```

#include <stdio.h>

int main(){

    int n, i, j;
    int graph[10][10];
    int peri = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for(i = 0; i < n; i++){

        for(j = 0; j < n; j++){

            scanf("%d", &graph[i][j]);
        }
    }

    for(i = 0; i < n; i++){

        for(j = 0; j < n; j++){

            if(graph[i][j] != 0 && i < j){

```

```

        peri = peri + graph[i][j];
    }
}

printf("\nTotal Edge Weight = %d\n", peri);

return 0;
}

```

5 Graph Perimeter Method 2 (Divide by 2)

```

#include <stdio.h>

int main(){

    int n, i, j;
    int graph[10][10];
    int sum = 0;

    printf("Enter number of vertices: ");
    scanf("%d", &n);

    printf("Enter adjacency matrix:\n");

    for(i = 0; i < n; i++){

        for(j = 0; j < n; j++){

            scanf("%d", &graph[i][j]);

            if(graph[i][j] != 0){

                sum += graph[i][j];
            }
        }
    }

    printf("\nTotal Edge Weight = %d\n", sum / 2);

    return 0;
}

```